React Shape Editor PRD

Overview

React Shape Editor is a lightweight browser-based design tool built in React that allows users to create and edit simple graphic objects on a canvas. The product is intended to satisfy a short technical exercise focused on editor-like interaction, clean React structure, sensible scope, and responsible use of AI coding assistance.[cite:25]

The target experience is closer to a minimal object editor such as a simplified slide or diagram tool than to Photoshop or full Figma. The product should feel intentional, responsive, and complete while remaining small enough to finish to a high standard within the exercise scope.[cite:25]

Problem Statement

The exercise requires a web interface where users can perform simple object modification on on-screen graphic elements implemented in React, including actions such as move, resize, recolor, and rotate.[cite:25]

The challenge is not only to implement these interactions, but to do so with clear structure, realistic scope, and enough polish to demonstrate understanding of the shipped code rather than assembling an over-scoped prototype.[cite:25]

Product Goals

- Build a micro design tool in React for editing simple graphic objects on a canvas.[cite:25]
- Deliver reliable core interactions in a normal desktop browser.[cite:25]
- Present a clear and intentional UI with sensible controls and visible feedback.[cite:25]
- Demonstrate clean React component structure and state handling for editor-like behavior.[cite:25]
- Keep the feature set intentionally scoped to avoid fantasy features that reduce quality or completeness.[cite:25]

Non-Goals

The following are explicitly out of scope for the first submission because they exceed the intended scale of a lightweight micro design tool or introduce unnecessary implementation risk:[cite:25]

- Image upload and image editing.
- Photoshop-style filters, masking, brush tools, or raster editing.
- Collaboration or cloud sync.

- Multi-select, grouping, and nested layers.
- Infinite canvas, rulers, advanced snapping systems, or artboards.
- Full export pipeline beyond an optional simple image export.
- Mobile-first interaction support.

Target Users

The primary user is an evaluator reviewing whether a candidate can build a small but real editing interface quickly in React while showing sound technical judgment. The app should therefore optimize for clarity, predictable interactions, and visible implementation quality rather than broad feature count.[cite:25]

User Experience Principles

- The UI should feel deliberate rather than decorative, with a clean dark-themed application shell and a clear white canvas area for editing.[cite:25]
- Controls should be easy to discover, with shape creation actions separated from property editing actions.[cite:25]
- Every important action should produce immediate visual feedback, especially selection, creation, movement, resizing, rotation, zoom changes, and property edits.[cite:25]
- When overlapping objects exist, behavior should remain predictable through consistent topmost selection and simple layer controls.
- The canvas should support navigation controls that feel natural in desktop editing software, including panning and zooming.[web:37] [web:41] [web:43]

Visual Design

The interface uses a monochrome dark theme. The application background should be dark gray with a subtle grid or dotted pattern, while the editable canvas should sit prominently in the center as a white rounded rectangle.

The layout consists of:

- A static vertical toolbar on the left for selection and shape creation tools.
- A static horizontal property bar at the top for editing the selected object's properties.
- A central canvas area for creation and manipulation.
- A visible zoom indicator with adjacent zoom controls placed near the top bar or canvas edge.[web:43]

This structure supports the requirement for clear layout and sensible editor-like interaction.[cite:25]

Supported Objects

The first release should support the following object types:

- Rectangle
- Ellipse
- Text

A line object may be added only if the core interactions above are fully stable. Polygon support is deferred because it increases geometry complexity without materially improving demonstration value for this exercise.

Object Data Model

Each object should include a shared base structure:

- `id`
- `type`
- `x`
- `y`
- `width`
- `height`
- `rotation`
- `fillColor`
- `borderWidth`
- `borderColor`
- `zIndex`

Text objects additionally include:

- `text`

The render order may be implemented using either `zIndex` or array order, but the behavior must be consistent and explainable.

Functional Requirements

1. Create Objects

Users can choose a creation tool from the left toolbar and create an object by dragging on the canvas. The object's position and size are determined by the drag start and drag end points.

Minimum creation tools:

- Rectangle tool
- Ellipse tool

- Text tool

Optional:

- Line tool

When an object is being created, the UI should show a live preview so the interaction feels intentional and responsive.[cite:25]

2. Select Objects

Users can select an object using the selection tool and left-clicking on the canvas. If objects overlap, the object with the highest visual layer should be selected first.

Clicking on empty canvas space should clear the current selection.

3. Move Objects

When an object is selected, the user can drag the object body to move it around the canvas. Position changes should update the object state immediately and be reflected in the property bar.

4. Transform Objects

When an object is selected, a visible transform frame should appear around it. The transform frame consists of:

- A bright blue rectangular outline around the selected object's bounds.
- Eight white resize handles positioned at the four corners and four edge midpoints.
- A short rotation stem extending above the object with a circular rotation handle.

Supported transformations:

- Dragging resize handles changes width and height.
- Dragging the rotation handle changes rotation in degrees.
- Dragging the object body moves the object.

Shift-assisted transforms:

- Holding Shift while corner-resizing preserves the object's original aspect ratio.
- Holding Shift while rotating snaps the angle to 45-degree increments.

Rotation should occur around the object's center unless otherwise documented.

5. Edit Object Properties

When an object is selected, the top property bar should display editable fields for the supported properties.

Shared editable properties:

- X position
- Y position
- Width
- Height
- Rotation
- Fill color
- Border width
- Border color

Text-only editable property:

- Text content

Layer controls:

- Bring forward
- Send backward

A simple swap with the nearest adjacent visual layer is sufficient. A full layer panel is not required.

6. Canvas Navigation

The canvas should support viewport navigation in addition to object editing. These controls are intended for desktop mouse usage and should not interfere with the normal left-click select and transform flow. Middle mouse drag is a common panning pattern in graphics tools, and scroll-wheel zoom is a standard interaction when building canvas-based editors [web:37] [web:43]. Zooming relative to the pointer position provides a more natural editing experience than zooming only toward the center [web:41].

Required canvas navigation behavior:

- Hold the middle mouse button and drag to pan the canvas view.
- Use the mouse scroll wheel to zoom in and out.
- Display the current zoom percentage in the interface.
- Provide `+`, `-`, and `=` buttons beside the zoom indicator to zoom in, zoom out, and reset zoom to 100 percent.

The zoom controls may be placed near the canvas or within the top bar, but they should remain clearly visible during editing. Reset behavior should return only the zoom level to 100 percent unless an expanded viewport reset is intentionally implemented.[web:43]

7. Visual Feedback

The product must show clear feedback during object interaction, consistent with the exercise requirements.[cite:25]

Required feedback includes:

- Visible selection state.
- Resize and rotation handles only when an object is selected.
- Immediate visual update during drag, resize, rotate, zoom, pan, and property edits.
- Clear cursor changes where appropriate.
- Property bar values updating to match the selected object.
- A visible zoom percentage indicator that stays in sync with canvas scale.

Interaction Rules

- Selection prioritizes the topmost overlapping object.
- Clicking empty canvas deselects.
- Tool choice determines whether the user is creating or selecting.
- Middle mouse drag pans the canvas without changing the current tool.[web:37] [web:43]
- Scroll wheel zoom adjusts the canvas view without changing object data.[web:41] [web:43]
- Objects remain constrained to a reasonable editing experience; freeform behavior that causes visual breakage should be avoided.
- Desktop mouse interaction is the primary target; touch support is not required for the first release.[cite:25]

Technical Requirements

- Built in React.
- Use a clean component structure with separation between canvas rendering, toolbar actions, property editing, and shape state management.[cite:25]
- Use predictable state handling suitable for editor-like behavior.[cite:25]
- Core interactions must work in a normal desktop browser, and any browser-specific issues must be documented in the README.[cite:25]

Recommended component structure:

- `App.tsx` for overall layout and shared state
- `Canvas.tsx` for the drawing and interaction surface
- `Toolbar.tsx` for tool selection
- `PropertyBar.tsx` for selected object controls
- `ShapeRenderer.tsx` for rendering objects
- `types/shapes.ts` for object definitions

- `utils/` for transformation and geometry helpers

Browser Scope and Limits

The product is expected to work in a normal desktop browser, which should be interpreted as current desktop Chrome or similar Chromium-based browsers first.[cite:25]

Potential limits to document if encountered:

- Minor differences in pointer, wheel, or transform behavior across browsers.
- Reduced support for touch devices.
- Any edge cases in text measurement or rotated resize behavior.
- Trackpad and mouse-wheel sensitivity differences that affect zoom feel.

Must-Have Release Scope

The first acceptable submission must include:

- Dark themed editor shell with left toolbar, top property bar, and central canvas.
- Rectangle and ellipse creation.
- Text creation if feasible within the same quality bar.
- Selection, move, resize, recolor, and rotate.
- Middle-mouse panning and scroll-wheel zoom.
- Visible zoom controls with percentage indicator and `+`, `-`, `=` actions.
- Property editing for selected objects.
- Simple layer ordering controls.
- Clear visual feedback for interaction states.
- Stable behavior in a desktop browser.[cite:25]

Nice-to-Have Scope

The following can be added only after the must-have scope is stable:

- Text object support if not included in the minimum build.
- Line object support.
- Simple duplicate or delete actions.
- Basic keyboard shortcuts such as Delete or arrow nudging.
- Optional export as image.
- Optional demo GIF for README.

Success Criteria

The submission is successful if it demonstrates the following outcomes:

- Users can create and modify graphic objects directly on the canvas.[cite:25]
- The interface feels more substantial than a static page while remaining clearly below full Figma complexity.[cite:25]
- The React structure and state model are easy to understand and defend in discussion.[cite:25]
- The scope appears intentional and finishable rather than inflated with incomplete features.[cite:25]
- The repository documents supported object types, known issues, setup steps, and report sections required by the exercise.[cite:25]

Deliverables Alignment

To align with the exercise, the repository should include:

- A public GitHub repository.
- A README with clone, install, and run instructions.[cite:25]
- A section listing supported object types and their allowed modifications.[cite:25]
- A known issues section.[cite:25]
- A markdown report with at least the following sections: Problem Statement, Methodology, Evaluation Dataset if applicable, Evaluation Methods if applicable, and Experimental Results.[cite:25]
- A brief note stating which AI tools were used and what was manually verified.[cite:25]

Risks and Trade-Offs

The highest implementation risks are rotated resizing, shape-specific geometry edge cases, and over-expanding the object model. To preserve quality, the project should favor fewer object types with reliable interactions over a broader set of partially working features.[cite:25]

This trade-off is aligned with the exercise's emphasis on scope judgment and evidence of understanding rather than sheer feature count.[cite:25]